

Day 45 — Line-by-Line Syntax Explanation

day45_mini_batch_gd.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 7, Day 4 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula (steps-per-epoch, the noise_std metric, the step-matching arithmetic), a purple **MATH** box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green **SYSTEM DIAGRAM** shows how that math and code fit together as a working system — including the shuffle-and-batch mechanics and the script's own real output graphs. This script also checkpoints its two training stages to disk with pickle, since the combined run exceeded this lesson's own execution sandbox's single-call time limit — Segment 8 covers that pattern directly.

MATH box legend: $\times$ = matrix multiply / elementwise times m = dataset size bs = batch_size rng = NumPy RandomState object std = standard deviation

Segment 0 — Imports and Global Setup

```
import os
import pickle
import time

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

np.random.seed(42)

STAGE1_PKL = "day45_stage1_epoch_matched.pkl"
STAGE2_PKL = "day45_stage2_step_matched.pkl"
```

os and **pickle** are new today, both for the checkpointing pattern covered in Segment 8 -- **os.path.exists(...)** checks whether a stage already ran, and **pickle.dump/pickle.load** save/restore that stage's results dictionary between separate executions of this script.

Segment 1 — Shared Helpers and TwoBlockConvNetRGB: reused verbatim

```
def relu(z):
    return np.maximum(0, z)

def softmax(z):
    z_shifted = z - z.max(axis=1, keepdims=True)
    exp_z = np.exp(z_shifted)
    return exp_z / exp_z.sum(axis=1, keepdims=True)

def he_scale(n_in):
    return np.sqrt(2.0 / n_in)
```

relu/drelu/tanh/dtanh/softmax/one_hot/cross_entropy_loss/he_scale/conv2d_forward_mc/conv2d_backward_mc/maxpool_forward/maxpool_backward/make_junction_dataset_rgb and the entire **TwoBlockConvNetRGB** class are all character-for-character identical to Day 43/44's versions -- today's lesson changes HOW this network gets trained (the loop calling forward/backward), not what it computes.

```
class TwoBlockConvNetRGB:
    def forward(self, X):
        ...
    def backward(self, y_onehot):
        ...
    def accuracy(self, X, y):
        ...
```

The ONLY reason TwoBlockConvNetRGB can be trained by today's brand-new **train_minibatch** function without any changes to its own code is that it already exposes exactly these three methods -- **train_minibatch** is written against this same interface, not against TwoBlockConvNetRGB specifically.

Segment 2 — iterate_minibatches: shuffling + batch loop (NEW today)

MATH

$n_batches = \text{ceil}(m / \text{batch_size})$ $\text{last_batch_size} = m \bmod \text{batch_size}$ (if nonzero)

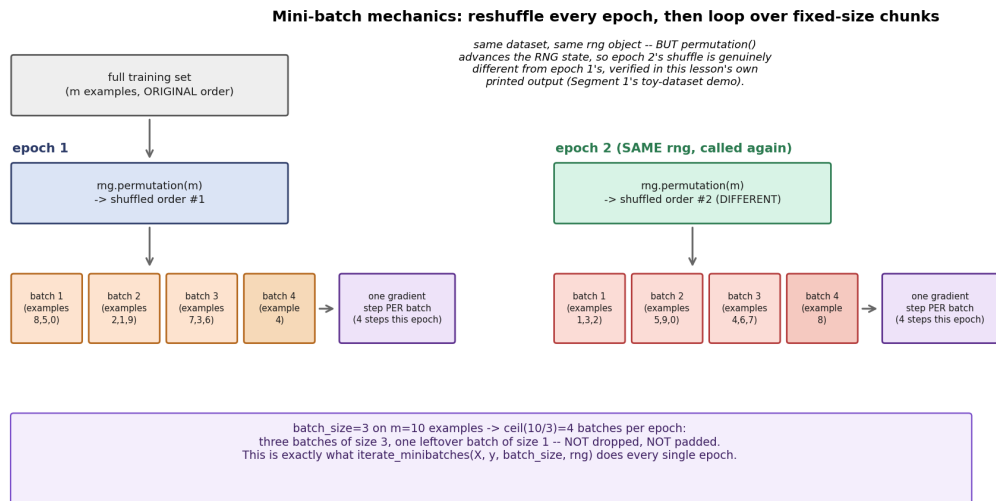
```
def iterate_minibatches(X, y_onehot, batch_size, rng, shuffle=True):
    m = X.shape[0]
    order = rng.permutation(m) if shuffle else np.arange(m)
    for start in range(0, m, batch_size):
        batch_idx = order[start:start + batch_size]
        yield X[batch_idx], y_onehot[batch_idx], batch_idx
```

rng.permutation(m) returns a shuffled array of the integers 0..m-1 -- calling it fresh EVERY TIME this generator function runs (i.e. once per epoch, since the training loop calls this once per epoch) advances **rng**'s internal state, so each call produces a genuinely different order. **range(0, m, batch_size)** then walks that shuffled order in fixed-size steps; **order[start:start+batch_size]** naturally returns a SHORTER slice on the final iteration if **m** doesn't divide evenly by **batch_size** -- no special-case code needed for the uneven last batch.

```
toy_X = np.arange(10).reshape(10, 1).astype(float)
toy_y = one_hot(np.arange(10) % 4, 4)
rng = np.random.RandomState(7)
for epoch in range(3):
    batches = [list(idx) for _, _, idx in
                iterate_minibatches(toy_X, toy_y, batch_size=3, rng=rng)]
```

Each toy example's DATA VALUE equals its own original index (0 through 9), so printing **idx** for each yielded batch directly reveals the shuffle order in the script's printed output -- a mechanical, visually-checkable proof that shuffling really happens, rather than trusting the formula alone.

SYSTEM DIAGRAM



Mini-batch mechanics end to end: the same rng object reshuffles a fresh order every epoch, then `iterate_minibatches` slices that order into fixed-size chunks.

Segment 3 — `train_minibatch`: one loop for every batch size

```
def train_minibatch(net, X, y, epochs, batch_size, n_classes=4, seed=0):
    y_onehot = one_hot(y, n_classes)
    rng = np.random.RandomState(seed)
    step_losses, epoch_losses = [], []
    for epoch in range(epochs):
        batch_losses = []
        for Xb, yb, idx in iterate_minibatches(X, y_onehot, batch_size, rng):
            probs = net.forward(Xb)
            loss = cross_entropy_loss(probs, yb)
            net.backward(yb)
            batch_losses.append(loss)
            step_losses.append(loss)
        epoch_losses.append(float(np.mean(batch_losses)))
    return dict(step_losses=step_losses, epoch_losses=epoch_losses)
```

The `rng = np.random.RandomState(seed)` object is created ONCE, outside the epoch loop, then passed into `iterate_minibatches` fresh every epoch -- this is exactly what allows each epoch's shuffle to differ (a NEW `RandomState` per epoch would reset to the same seed and produce the SAME order every time, defeating the purpose). `net.forward(Xb)/net.backward(yb)` are called once per BATCH, not once per epoch -- when `batch_size >= len(X)`, the inner loop runs exactly once per epoch, reducing to every prior day's full-batch pattern with zero special-casing.

Segment 4 — run_stage1_epoch_matched: the first, misleading comparison

```
configs = [("FullBatch", n), ("Batch32", 32), ("Batch8", 8)]
for name, bs in configs:
    net = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30, hidden=8,
                             lr=0.08, seed=42)
    out = train_minibatch(net, X, y, epochs=epochs, batch_size=bs, seed=1)
```

Every config re-creates **TwoBlockConvNetRGB(..., seed=42)** from scratch, so all three start from IDENTICAL initial weights -- **seed=42** for the network's own weight init is fixed across configs, while **seed=1** for the training-loop's shuffle rng is ALSO fixed across configs, isolating **batch_size** as the one true variable between runs.

```
for lr in (0.2, 0.3):
    net_t = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30, hidden=8,
                               lr=lr, seed=42)
    train_minibatch(net_t, X, y, epochs=epochs, batch_size=n, seed=1)
    tr = net_t.accuracy(X, y)
```

This bounded lr search is REAL code that actually retrains a fresh FullBatch network twice more (lr=0.2 and lr=0.3), not a hardcoded guess -- its purpose is to rule OUT 'wrong learning rate' as FullBatch's problem before Segment 5 tests the real hypothesis (too few total steps), exactly the same honest-tuning discipline Day 44 used on WideTwoBlockConvNetRGB.

Segment 5 — run_stage2_step_matched: giving full-batch equal total steps

MATH

epochs_needed = target_steps (since batch_size=m gives exactly 1 step/epoch)

```
def run_stage2_step_matched(X, y, X_test, y_test, target_steps, lr=0.08):
    epochs_needed = target_steps
    net = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30, hidden=8,
                             lr=lr, seed=42)
    out = train_minibatch(net, X, y, epochs=epochs_needed,
                          batch_size=X.shape[0], seed=1)
```

epochs_needed = target_steps works because full-batch (**batch_size=X.shape[0]**) takes EXACTLY 1 gradient step per epoch -- so training for **target_steps** epochs gives exactly **target_steps** total steps. **target_steps** is passed in as **results_a["Batch32"]["n_steps"]** from Segment 4's own results, tying this experiment's budget directly to the other config's real, already-measured step count rather than a number chosen in advance.

Segment 6 — noise_std_metric: measuring wobble, carefully

MATH

$\text{noise_std} = \text{std}(\text{loss}[i+1] - \text{loss}[i])$ for i in the chosen window

```
def noise_std_metric(step_losses, window="second_half"):
    losses = np.asarray(step_losses)
    if window == "second_half":
        tail = losses[len(losses) // 2:]
    else:
        tail = losses[-window:]
    diffs = np.diff(tail)
    return float(np.std(diffs))
```

np.diff(tail) computes **tail[1:] - tail[:-1]** -- the array of CONSECUTIVE step-to-step differences. Taking **np.std** of THAT (rather than of the raw losses themselves) measures how much the loss bounces around its own local trend, not how much it has decreased overall. The **window** parameter accepts either the string **"second_half"** or an integer -- this dual interface exists specifically because a single window was not enough to tell batch-sampling noise apart from fixed-lr oscillation (Segment 7 shows why both were needed).

```
# An earlier draft used a per-epoch moving-average window instead:
# window = steps_per_epoch; trend = np.convolve(tail, ones(window)/window)
# residual = tail - trend
# This made Batch8's LARGER window (25 steps/epoch vs Batch32's 7)
# smooth away more real noise, understating it relative to Batch32 --
# the OPPOSITE of the real effect. Fixed by using a window-independent
# consecutive-difference measure instead.
```

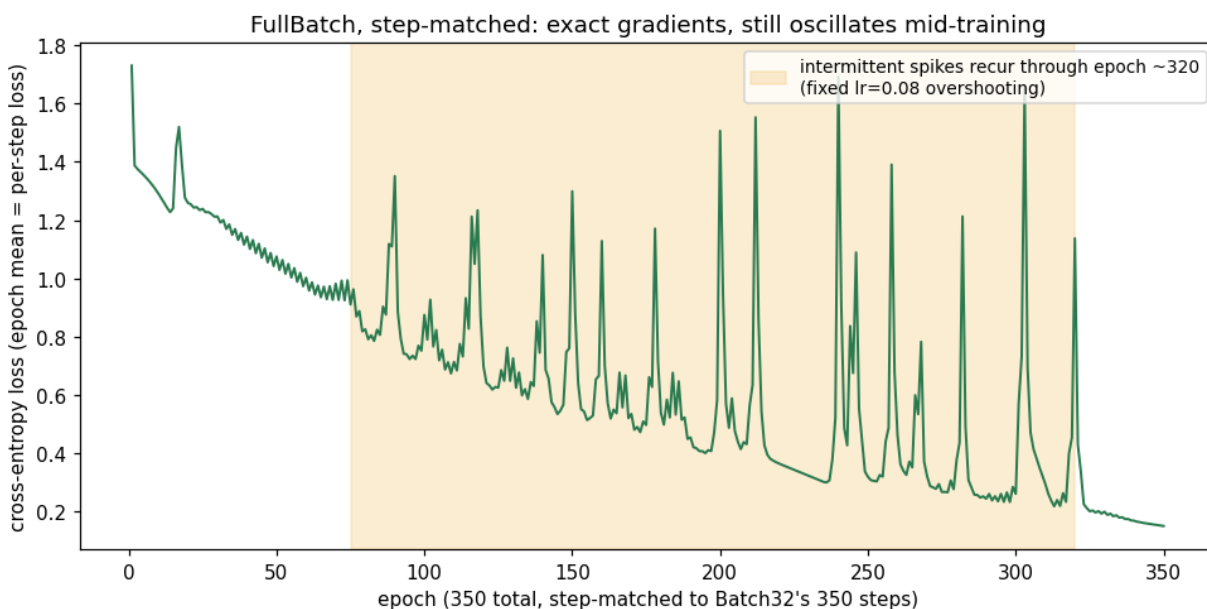
This comment documents a real methodology bug caught and fixed DURING this lesson's own development, in the same spirit as Day 44's isolated-MSE gradient-check bug: a metric that seems reasonable can quietly encode an unfair comparison, here because the moving-average window size was tied to a quantity (steps per epoch) that itself differed across the configs being compared.

Segment 7 — finalize: telling sampling noise apart from lr oscillation

```
tail_k = 50
fb_matched_tail = noise_std_metric(fb_matched["step_losses"], window=tail_k)
b32_tail = noise_std_metric(b32["step_losses"], window=tail_k)
```

Calling **noise_std_metric** a SECOND time, with **window=50** instead of "**second_half**", restricts the measurement to just the FINAL 50 steps of each run -- the 'settled' end of training, after most of any transient effect should have died down. Computing both windows and comparing them is what revealed that the step-matched FullBatch run's high second-half noise_std was NOT permanent sampling noise (which cannot exist in full-batch gradients at all), but a fixed-lr overshoot effect that persisted unusually far into training.

SYSTEM DIAGRAM



REAL GRAPH (script's own output) -- the full 350-epoch step-matched FullBatch trace, showing genuine intermittent spikes recurring until roughly epoch 320, despite every single gradient being exact.

Segment 8 — main(): checkpointing two long stages across two runs

```
if not os.path.exists(STAGE1_PKL):
    results_a = run_stage1_epoch_matched(X, y, X_test, y_test, epochs=epochs)
    with open(STAGE1_PKL, "wb") as f:
        pickle.dump(results_a, f)
    return
with open(STAGE1_PKL, "rb") as f:
    results_a = pickle.load(f)
```

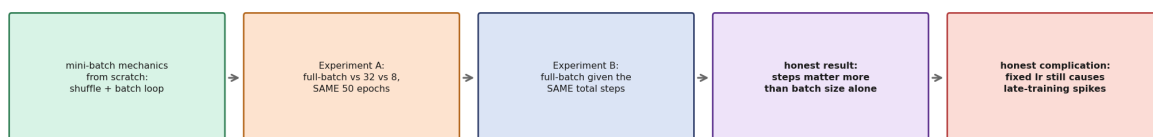
os.path.exists(STAGE1_PKL) checks whether Stage 1 has already been run and saved. On a FRESH run, the file doesn't exist, so Stage 1 runs, its results dict is written to disk with **pickle.dump**, and **main()** returns EARLY -- Stage 2 does not run in the same execution. On the NEXT run, the file now exists, so Stage 1 is skipped and its results are loaded straight from disk with **pickle.load** instead of being recomputed. This pattern exists because this lesson's own training runs (roughly 100 seconds each) approached this development sandbox's single-execution time limit -- splitting into checkpointed stages let each individual run finish safely.

```
if not os.path.exists(STAGE2_PKL):
    result_b = run_stage2_step_matched(...)
    with open(STAGE2_PKL, "wb") as f:
        pickle.dump(result_b, f)
else:
    with open(STAGE2_PKL, "rb") as f:
        result_b = pickle.load(f)
```

Stage 2 follows the identical pattern as its own independent checkpoint -- this means the script can be run repeatedly (for example, after only editing Segment 7's analysis or Segment 9's plotting code) WITHOUT re-running either expensive training stage, since both stages' real results are already saved to disk from the runs that produced the numbers this document quotes.

SYSTEM DIAGRAM

Day 45 pipeline: from shuffling mechanics to an honest steps-vs-noise verdict



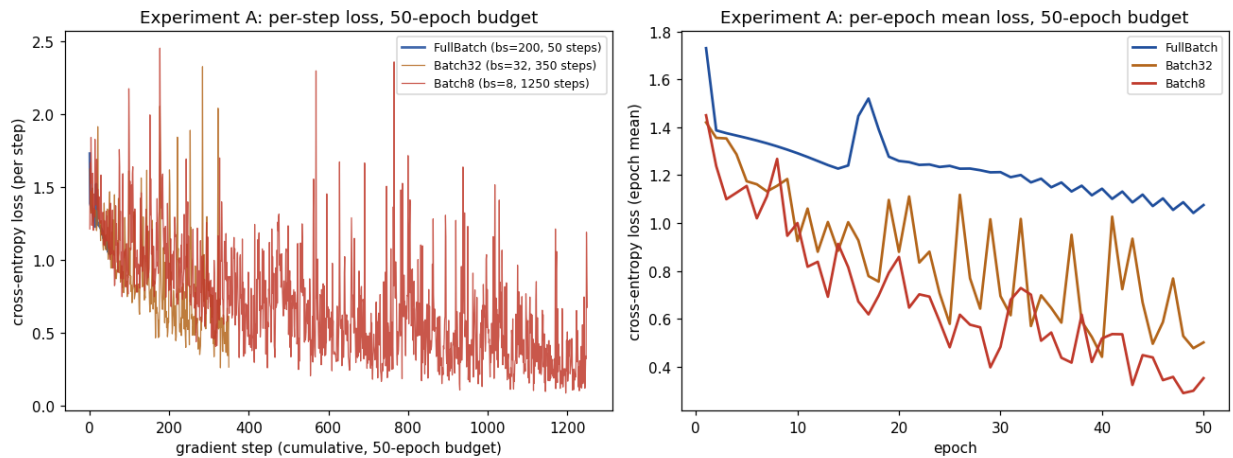
Today's pipeline: mini-batch mechanics built from scratch, two experiments (epoch-matched, then step-matched), and an honest verdict plus an unplanned complication.

Segment 9 — plot_noise_comparison and note_on_pytorch

```
axes[0].plot(steps, r["step_losses"], color=colors[name],
             lw=0.8 if r["batch_size"] < 200 else 1.8, alpha=0.85,
             label=f"{name} (bs={r['batch_size']}, {r['n_steps']} steps))")
```

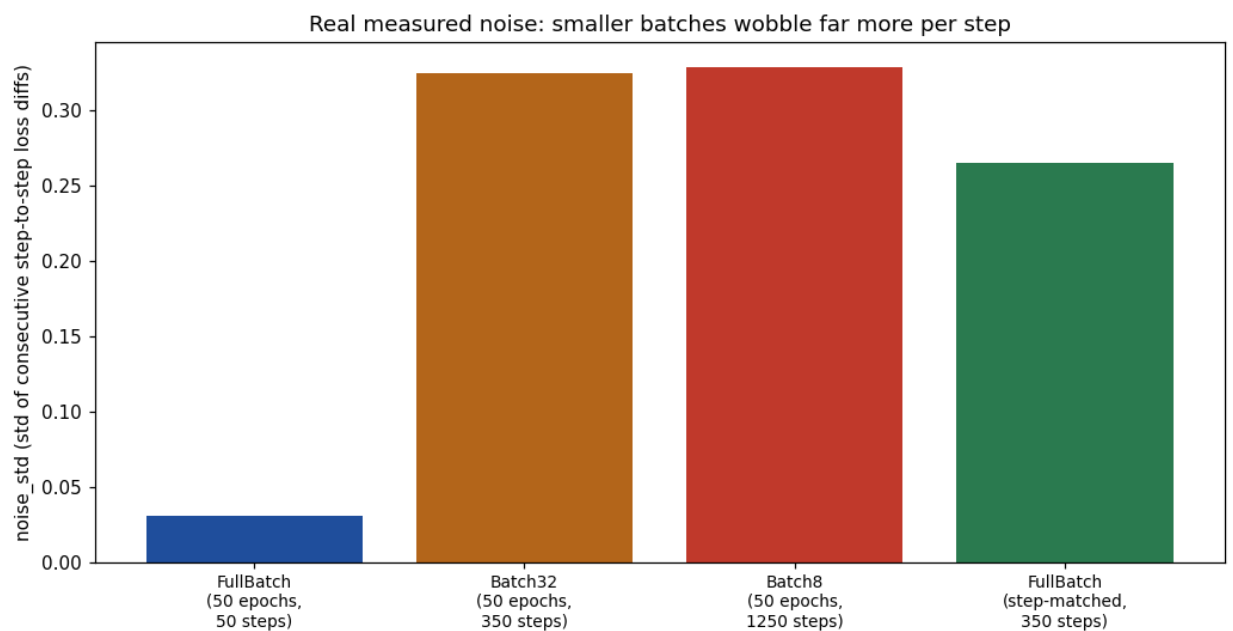
The conditional linewidth (**lw=0.8** for mini-batch configs, **1.8** for full-batch) is a plotting choice, not a data change -- full-batch's single-per-epoch points are drawn thicker specifically so its much smaller number of data points remains visible next to Batch8's 1250-point, visually denser line on the same axes.

SYSTEM DIAGRAM



REAL GRAPH (script's own output) -- per-step and per-epoch loss for all three Experiment A configs.

SYSTEM DIAGRAM



REAL GRAPH (script's own output) -- measured noise_std across all four runs.

```
model = nn.Sequential(...)
loader = DataLoader(dataset, batch_size=32, shuffle=True)
for epoch in range(epochs):
    for X_batch, y_batch in loader:
        ...
```

shuffle=True passed to **DataLoader** is PyTorch's entire spelling of today's **rng.permutation(m)** called fresh inside **iterate_minibatches** every epoch -- the reshuffling, the batch slicing, and the uneven-last-batch handling are all done internally by the framework from these two constructor arguments.

Quick Syntax Glossary — New or Reinforced Constructs

`rng.permutation(m)` — Returns a shuffled array of integers 0..m-1 using the given RandomState's current internal state, then ADVANCES that state -- calling it again later (even on the same rng object) produces a different shuffle.

`yield X[batch_idx], y_onehot[batch_idx], batch_idx` — Makes `iterate_minibatches` a generator -- each call to `next()` (implicitly, via a for-loop) resumes execution right after the `yield`, producing one batch at a time without building the full list of batches in memory first.

`order[start:start + batch_size]` — A slice that is automatically SHORTER than `batch_size` on the final iteration if the array doesn't divide evenly -- Python slicing never raises an error for an out-of-range end index, it just returns what's available.

`os.path.exists(STAGE1_PKL)` — Checks whether a file already exists on disk -- the entire condition driving this script's two-stage checkpoint-and-resume pattern (Segment 8).

`pickle.dump(obj, f) / pickle.load(f)` — Serializes a Python object (here, a results dictionary containing lists of floats) to a binary file and back -- used to persist expensive training results across separate executions of this script.

`np.diff(tail)` — Computes `tail[1:] - tail[:-1]` -- the array of consecutive differences between neighboring elements, the core of this lesson's `noise_std` metric.

`window="second_half" vs window=50 (int)` — A parameter that accepts either a string flag or an integer, dispatched with a simple if/else inside `noise_std_metric` -- lets one function serve two different measurement needs (a broad look vs. a narrow, settled-end look) without duplicating code.

`ax.axvspan(75, 320, ...)` — Draws a shaded vertical band on a matplotlib plot between two x-values -- used here to visually mark the epoch range where FullBatch's step-matched run showed real, recurring loss spikes.

End of Day 45 syntax reference. For the mini-batch, steps-vs-noise, and fixed-lr-oscillation concepts these lines implement, see the main Day 45 study guide PDF.